

COMPARISON OF HYBRID MODELS FOR DATA SECURITY: ENCRYPTION AND STEGANOGRAPHY USING IMAGES AND AUDIO

BY

ALIYU ABUBAKAR TUKUR

17/47719U/1

BEING AN UNDERGRADUATE PROJECT SUBMITTED TO THE COMPUTER SCIENCE DEPARTMENT, FACULTY OF SCIENCE, ABUBAKAR TAFAWA BALEWA UNIVERSITY BAUCHI, IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR THE AWARD OF B. TECH (HONS) IN COMPUTER SCIENCE

SEPTEMBER, 2024

DECLARATION

I Aliyu Abubakar Tukur, a student of the department of Computer Science with registration number 17/47719U/1, Abubakar Tafawa Balewa University Bauchi, hereby declare this project titled **"Comparison of Hybrid Models for Data Security: Encryption and Steganography Using Images and Audio"** has been carried out by me under the supervision of MR ISMAIL ALIYU and that it has never been submitted or presented for award of any degree in any form to any institution. All sources of material toward realizing this project are specifically acknowledged via the references provided.

CERTIFICATION

This project titled **"Comparison of Hybrid Models for Data Security: Encryption and Steganography Using Images and Audio"** written by Aliyu Abubakar Tukur a student of Computer science department with registration number 17/47719U/1 meets the requirements governing the award of the degree of Bachelor of Technology in Computer Science and is approved for its contribution to knowledge and literary representation.

---------- ----------

Mr. ISMAIL ALIYU Date
(Project Supervisor)

---------- ----------

Dr. FATIMA UMAR ZAMBUK Date
(Head of Department)

---------- ----------

Date

(External Supervisor)

ACKNOWLEDGEMENT

My appreciation also goes to the entire staff of Department of Computer Science for the required discipline I gained which actually shaped my ability towards this great achievement.

My profound gratitude goes to Almighty Allah who saw me through my Degree program and has enabled me to embark on this Project. He has been my all in all. I am grateful to my supervisor for checking on me to make sure that I was learning and doing the appropriate work needed to tackle the current world challenges as they come. My utmost gratitude goes to my entire family especially my older brother Engr. Adamu Abubakar Tukur who has been a backbone in my life and also a role model to me since childhood, following him is Geo. Sunusi Abubakar Tukur and Sir. Nasiru Abubakar Tukur who worked really hard to ensure that my life is a success, I appreciate you all. I am grateful to the Head of Department, Computer Science, Dr. Fatima Umar Zambuk for her advice and the encouragements she gave to students since the beginning of the journey. I am also grateful to my Departmental lecturers for being up and running to see that this degree program became a success. May God replenish your energy and bless you all.

DEDICATION

I dedicate this work to Almighty Allah who has been my source of inspiration, wisdom, guidance, knowledge and understanding. To Him I give all the glory, and also to my parents Alh. Abubakar Tukur and Hajiya Aishatu Abdulrahman and also my older brothers who have been there for me and helped me financially and morally.

ABSTRACT

The escalation in data breaches in recent years emphasize the need for stronger data protection mechanisms. This project introduces a hybrid model that integrates encryption with steganography to enhance the security and concealment of sensitive information. The system is tested on two mediums "image and audio" where encrypted data is embedded using steganography. A comparative analysis is conducted on the time taken for data embedding, file size before and after embedding, and the reliability of data extraction and decryption. The findings reveal that both image-based and audio-based steganography provide secure solutions for data transmission, but with notable tradeoffs in terms of speed, storage, and capacity. This hybrid approach offers an enhanced security model for protecting sensitive data in digital communications.

Table of Contents

DECLARATION....ii

CERTIFICATION....iii

ACKNOWLEDGEMENT....iv

DEDICATION....v

ABSTRACT....vi

Table of Contents....vii

CHAPTER ONE....1

INTRODUCTION....1

1.1 Background of the Study....1
1.2 Problem Statement....1
1.3 Aim and Objectives....2
CHAPTER TWO....4

LITERATURE REVIEW....4

2.1 Introduction....4
2.2 Data Security and Encryption....5
2.3 Steganography....5
2.4 Hybrid Models for Data Security....5
2.5 Image vs. Audio Steganography....6
CHAPTER THREE....7

METHODOLOGY....7

3.1 Introduction....7
3.2 Overview of the existing system....7
3.3 Problems Associated with the Existing System....8

3.4 Choice of Methodology....9
3.5 System Design....10
3.6 Tools and Technologies....11
3.7 AES Encryption....11
3.8 LSB Steganography....11
3.9 Performance Metrics....11
CHAPTER FOUR....13

IMPLEMENTATION AND RESULT....13

IMPLEMENTATION....13
4.1 AES Encryption class....13
4.2 Image steganography utility class....14
4.3 Audio steganography utility class....15
4.4 GUI implementation class....16
4.5 Hybrid model comparison class....19
RESULT....20
CHAPTER FIVE....21

SUMMARY, CONCLUSION AND RECOMMENDATION....21

5.1 Summary....21
5.2 Conclusion....22
5.2 Recommendations....22
REFERENCES....24

APPENDIX....25

Figure 1: The waterfall model. (Source: Summerville, 2011).....10

Figure 2: AES encryption class.....14

Figure 3: image steganography class.....15

Figure 4: Audio steganography class.....16

Figure 5: GUI implementation.....17

Figure 6: GUI implementation ii.....17

Figure 7: GUI implementation iii.....18

Figure 8: GUI implementation iv.....18

Figure 9: GUI implementation v.....19

Figure 10: hybridmodelcomparison main class.....19

Figure 11: running program.....20

CHAPTER ONE

INTRODUCTION

This project aims to develop and compare two hybrid models for data security by integrating encryption with steganography. The focus is on embedding encrypted data within image and audio files, evaluating the effectiveness, robustness, and practicality of each method. The comparison will be based on data capacity, perceptual impact, robustness to compression, and computational efficiency.

1.1 Background of the Study

With the increasing need for secure communication in today's digital world, it is essential to explore and implement advanced methods for protecting sensitive information. Cryptography and steganography are two key techniques used for data security. Cryptography ensures the confidentiality and integrity of data through encryption, while steganography hides the existence of the data itself. By combining these two techniques, we can create robust hybrid models that provide enhanced security.

This project explores the potential of combining Advanced Encryption Standard (AES) encryption with steganography to create a more robust security solution. By embedding encrypted data within cover files (images and audio), the hybrid model enhances both the confidentiality and the concealment of information during transmission.

1.2 Problem Statement

While encryption ensures the confidentiality of data, its presence can be easily detected, making it a target for interception and decryption attempts.

Existing data security methods either focus on encryption or steganography individually. There is a need to explore the combined use of these techniques to leverage their strengths and mitigate their weaknesses. This project seeks to develop and compare hybrid models that use encryption alongside image and audio steganography. Combining these two techniques provides a dual-layer defense. However, questions remain about which medium, image or audio offers better performance in terms of capacity, embedding speed, and data integrity.

1.3 Aim and Objectives

This project aims to develop and compare two hybrid models for data security by integrating encryption with steganography. This project aims to develop a hybrid model that integrates encryption and steganography for securing data and compares the performance of embedding data in images and audio files. The objectives are as follows:

To encrypt sensitive data using AES.

To embed the encrypted data into image (.png) and audio (.wav) files using Least Significant Bit (LSB) steganography.

To evaluate the embedding time, file size changes, and accuracy of data retrieval for both mediums.

To identify the advantages and limitations of each hybrid model.

1.4 Scope and Limitation

Scope:

Data Types: This project will focus on hiding textual data within images(.png) and audio(.wav) files.

Steganographic Techniques: The primary techniques used will be Least Significant Bit (LSB) for both images and audio.

Encryption Algorithm: Advanced Encryption Standard (AES) will be used for encrypting data.

Evaluation Metrics: Embedding time, file size variations, and accuracy of data extraction for both mediums will be evaluated.

Implementation: The project will be implemented in Java, utilizing available libraries for image and audio processing.

Comparison: The project will compare the two hybrid models (image-based and audio-based) on predefined metrics.

Limitations:

Simplicity of Steganographic Techniques: The project will use basic LSB steganography, which might not be the most robust method against sophisticated steganalysis.

File Formats: The project will be limited to PNG for images and WAV for audio files to avoid complexities associated with different file formats.

Computational Resources: The project will be conducted using standard computational resources available in a typical undergraduate setting, which might limit the scale of testing.

Scope of Data: The project will focus on hiding relatively small amounts of data, suitable for proof of concept rather than large-scale implementation.

Other encryption and steganographic techniques are outside the scope of this investigation. Additionally, the impact of different image or audio qualities is not considered in the performance evaluation.

CHAPTER TWO

LITERATURE REVIEW

2.1 Introduction

Cryptography is the practice of securing information by converting it into a format that is unreadable to unauthorized users. The primary goal of cryptography is to protect data confidentiality, integrity, and authenticity. Two widely used cryptographic algorithms are the Advanced Encryption Standard (AES) and the Rivest-Shamir-Adleman (RSA) algorithm.

AES (Advanced Encryption Standard) is a symmetric encryption algorithm standardized by the National Institute of Standards and Technology (NIST) in 2001. It operates on fixed block sizes of 128 bits and supports key sizes of 128, 192, and 256 bits. AES is known for its high performance and strong security, making it suitable for a wide range of applications, from securing online transactions to protecting sensitive government information. AES encryption is computationally efficient, which is critical for real-time applications.

Cryptography is foundational for securing data, but encrypted data can still be susceptible to interception. This is where steganography comes into play, hiding the existence of the data itself.

Steganography is the art and science of hiding information within other, seemingly innocuous, data. Unlike cryptography, which obscures the content of a message, steganography conceals the fact that a message exists. Various steganographic techniques have been developed for embedding data within digital media, such as images and audio files.

2.2 Data Security and Encryption

Data security involves the protection of data from unauthorized access, breaches, or corruption. Encryption plays a fundamental role in this process by transforming readable data into an unreadable format using cryptographic algorithms. AES (Advanced Encryption Standard) is one of the most secure and widely used encryption algorithms due to its high performance and resilience against attacks (Stallings, 2016). It is a symmetric key algorithm that is used for securing sensitive information in digital communications.

2.3 Steganography

Steganography refers to the process of hiding secret information within a cover medium, such as images or audio files, in a way that prevents detection by unauthorized users. The Least Significant Bit (LSB) method is one of the simplest and most commonly used techniques in image and audio steganography. In this method, the least significant bits of the cover file are altered to embed the secret data. This change is imperceptible to the human eye or ear, allowing data to be concealed within the cover medium (Johnson & Jajodia, 1998).

2.4 Hybrid Models for Data Security

A hybrid security model that combines encryption and steganography offers a more robust solution for secure data transmission. The encrypted data is first transformed into ciphertext, and then the ciphertext is hidden within an image or audio file using steganographic techniques. This double-layered protection ensures that even if the hidden data is discovered, it remains encrypted, further protecting it from unauthorized access. Research has shown that hybrid models provide enhanced security compared to using encryption or steganography alone (Almeida et al., 2021).

2.5 Image vs. Audio Steganography

Image and audio steganography each have their strengths and weaknesses. Images are more widely used as cover media in steganography because of their larger capacity and the ability to embed data with minimal visual distortion. Audio steganography, on the other hand, has the advantage of embedding data in sound files, making it harder to detect by human senses. However, embedding data in audio files is often more computationally intensive due to the complexity of audio data (Chen et al., 2022). Recent studies suggest that while image steganography is faster and more efficient, audio steganography offers higher levels of imperceptibility (Singh et al., 2022).

CHAPTER THREE

METHODOLOGY

3.1 Introduction

In today's digital age, ensuring the security and confidentiality of sensitive information has become a paramount concern. With the rise of sophisticated cyber threats and an ever-growing reliance on data exchange, traditional security measures are often deemed inadequate. To address this challenge, a comprehensive and robust approach to data security is essential. The integration of cryptography and steganography in hybrid models offers a promising approach to secure data transmission. By combining these techniques, it is possible to create robust security solutions that protect both the content and existence of sensitive information. This project aims to explore and compare two such hybrid models, focusing on the use of image and audio steganography alongside encryption.

3.2 Overview of the existing system

Existing systems in cryptography and steganography provide tools and techniques for data security. Cryptography systems include AES, RSA, ECC, and PGP, offering strong encryption and secure key exchange. Steganography systems hide data within digital media files. OpenStego supports multiple algorithms and hides data in images, audio, and video files. StegHide specializes in hiding information in image and audio files, with encryption and compression options. OutGuess focuses on steganography within images while preserving their quality. Invisible Secrets offers a comprehensive suite of security features, including encryption, steganography, file shredding, and password management.

3.3 Problems Associated with the Existing System

The existing systems address various security needs but face challenges like key management, algorithm weaknesses, implementation flaws, steganalysis, embedding limitations, security through obscurity, and usability/integration.

Key Management: The management of encryption keys in cryptography systems can be complex and prone to vulnerabilities if not implemented correctly. Weak key generation, inadequate key storage, or improper key distribution can undermine the overall security of the system.

Algorithm Weaknesses: Cryptographic algorithms used in existing systems may become vulnerable over time due to advancements in computational power and new attack techniques. Outdated or weakened algorithms can be susceptible to brute-force attacks or cryptanalysis, compromising the security of the system.

Implementation Flaws: Even if the cryptographic algorithms themselves are secure, implementation flaws can introduce vulnerabilities. Poorly coded software, misconfigurations, or insufficient testing can create loopholes that attackers can exploit, bypassing the intended security measures.

Steganalysis: Steganalysis refers to the detection of hidden data within steganographic carriers Advanced analysis techniques and machine learning algorithms have been developed to detect steganography, making it essential for steganography systems to employ robust and advanced hiding techniques that can withstand detection attempts.

Limited Embedding Capacity: Steganography techniques often have limitations on the amount of data that can be concealed within a carrier file. High embedding capacity can lead to detectable

changes in the cover file, while low embedding capacity may restrict the usefulness of the steganography system for practical purposes.

Security through Obscurity: Some steganography systems may rely on the secrecy or obscurity of the hiding technique itself for security. However, relying solely on obscurity can be risky, as once the technique is exposed or reverse-engineered, the hidden data becomes vulnerable.

Usability and Integration: The usability and integration of cryptography and steganography systems can present challenges. Complex user interfaces, lack of interoperability between different systems, or limited compatibility with existing software can hinder their adoption and effectiveness.

Addressing these problems requires continuous research and development, adherence to best practices, regular updates and patches, and strong collaboration between security experts and system developers. Striving for stronger key management practices, more robust algorithms, rigorous implementation reviews, and improved usability and integration can enhance the overall security of these systems.

3.4 Choice of Methodology

The waterfall methodology will be adopted in this work because of its simplicity to a relative extend. It is normally used when the requirements are well understood and unlikely to change radically during system development, and this is the case in this study, hence the need to adopt it especially for a novice.

The waterfall model as indicated earlier is a sequential design process in which progress is seen as flowing steadily downwards like waterfall through the phases of conception, initiation, analysis, design, construction, testing, implementation and maintenance.

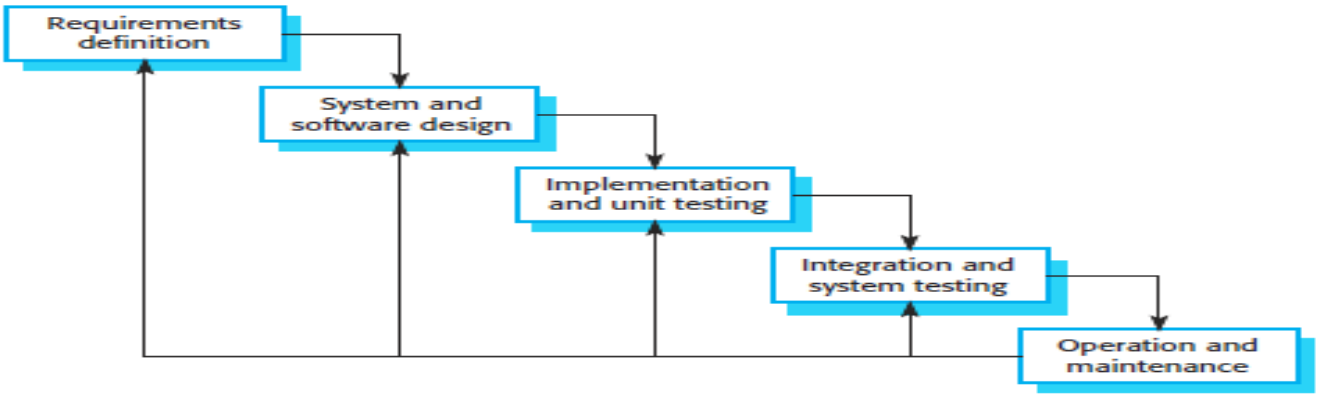


Figure 1: The waterfall model. (Source: Summerville, 2011)

More specifically, the motivation to use this model was premised on the following factors:

i. The methodology is relatively simple to understand and easy to implement.

ii. It is normally used when the requirements are well understood and unlikely to change radically during system development, and this is the case in this study.

iii. This model is best suited here because available literature shows that developers already have designed and developed assorted software products using it in the past and are aware of all its domains, and the methodology is still widely used today.

iv. It reinforces good habits: define-before- design, design-before-code.

v. It identifies deliverables and milestones.

vi. Works well on mature products and weak teams.

3.5 System Design

The system is designed to encrypt data using AES encryption and then embed the encrypted data into either an image (.png) or audio file (.wav) using LSB steganography. The system flow is as follows:

1. Input data is provided by the user for encryption.
2. The data is encrypted using AES encryption.

3. The encrypted data is embedded into an image or audio file using LSB steganography.
4. The system extracts the embedded data and decrypts it to verify accuracy.

3.6 Tools and Technologies

- **Programming Language**: Java
- **Tools**: Java Development Kit (JDK), Java libraries for image and audio processing.
- **Libraries**: Java Cryptography Extension (JCE) for AES encryption, Java Swing for the graphical user interface, javax.crypto for encryption, javax.imageio for image processing, and javax.sound.sampled for audio processing.
- **File Types**: .png for images and .wav for audio files.

3.7 AES Encryption

AES encryption is a symmetric key encryption algorithm that transforms the plaintext data into an unreadable ciphertext format. In this project, a 128-bit AES encryption key is used to secure the data.

3.8 LSB Steganography

The Least Significant Bit (LSB) method is used for embedding the encrypted data into both image and audio files. In images, the encrypted data is embedded in the least significant bits of the pixel values. In audio files, the encrypted data is embedded in the least significant bits of the audio sample values.

3.9 Performance Metrics

The system's performance is evaluated based on the following metrics:

- ✓ **Data capacity**

- ✓ **Embedding Time**: The time taken to embed the encrypted data into the image or audio file.
- ✓ **File Size Variations**: The difference in file size before and after data embedding.
- ✓ **Data Retrieval Accuracy**: The success rate of extracting and decrypting the data from the cover file.

CHAPTER FOUR

IMPLEMENTATION AND RESULT

IMPLEMENTATION

4.1 AES Encryption class

Utilizes the following java libraries;

- ✓ Javax.crypto.cipher
- ✓ javax.crypto.KeyGenerator
- ✓ javax.crypto.SecretKey.

and also consist of three methods;

1. The encrypt method which:
    - ✓ Create an AES cipher instance
    - ✓ Initialize cipher in encrypt mode with the secret key
    - ✓ Encrypt the data and return it.
2. The decrypt method which:
    - ✓ Create an AES cipher instance
    - ✓ Initialize cipher in decrypt mode with the secret key
    - ✓ Decrypt the data and return it.
3. The generatekey method which:
    - ✓ Create a key generator for AES
    - ✓ Initialize the key generator with a key size of 128 bits
    - ✓ Generate and return the AES secret key.

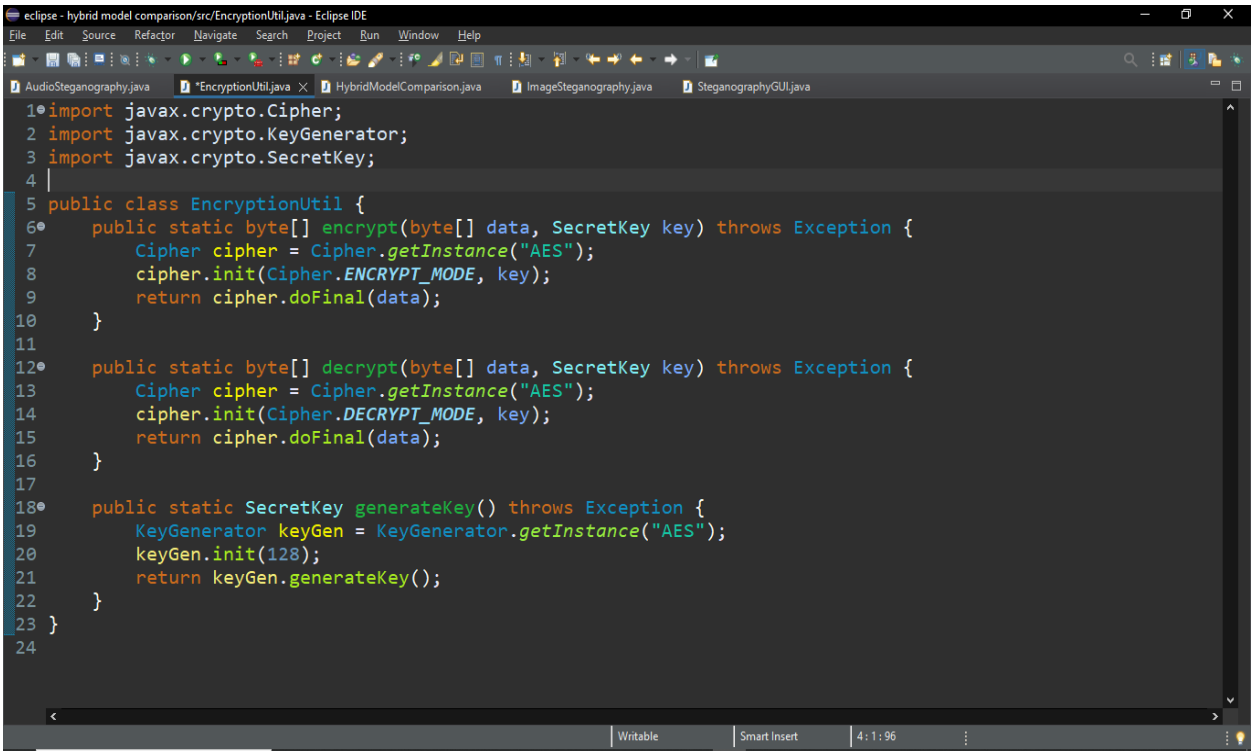


Figure 2: AES encryption class

4.2 Image steganography utility class

Utilizes the following java libraries;

java.awt.image.BufferedImage; Import for handling images

java.io.File; For file handling

javax.imageio.ImageIO; Import for reading/writing images

java.nio.file.Files; For file operations

java.nio.file.Paths; Import for file path operations

and consist of two methods:

1. The embed method: which takes in image and data as parameters and performs embedding operation.

2. The extract method: which also takes in the stego image and perform extraction operation.

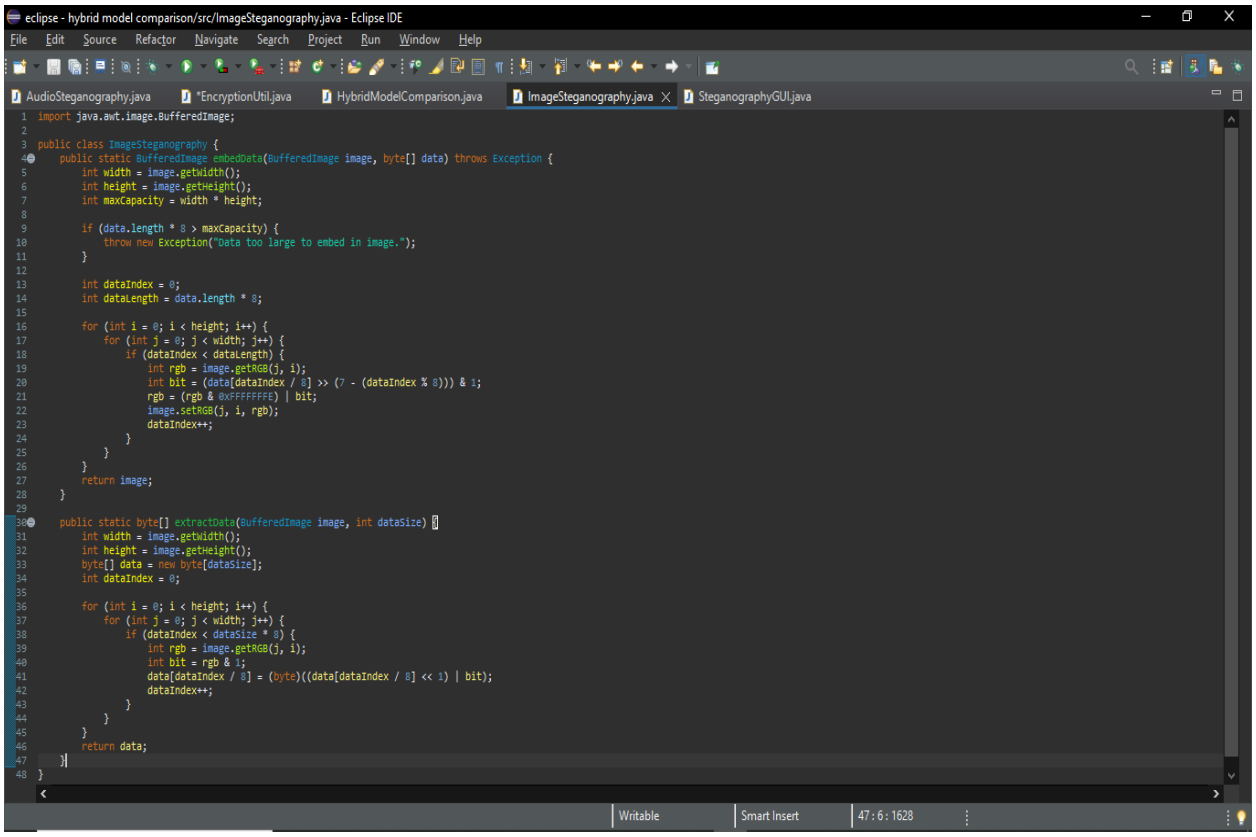


Figure 3: image steganography class

4.3 Audio steganography utility class

Utilizes the following java libraries;

java.io.File; For file handling

java.nio.file.Files; For file operations

java.nio.file.Paths; Import for file path operations

and consist of two methods:

1. The embed method: which takes in audio and data as parameters and performs embedding operation.

2. The extract method: which also takes in the stego audio and perform extraction operation.

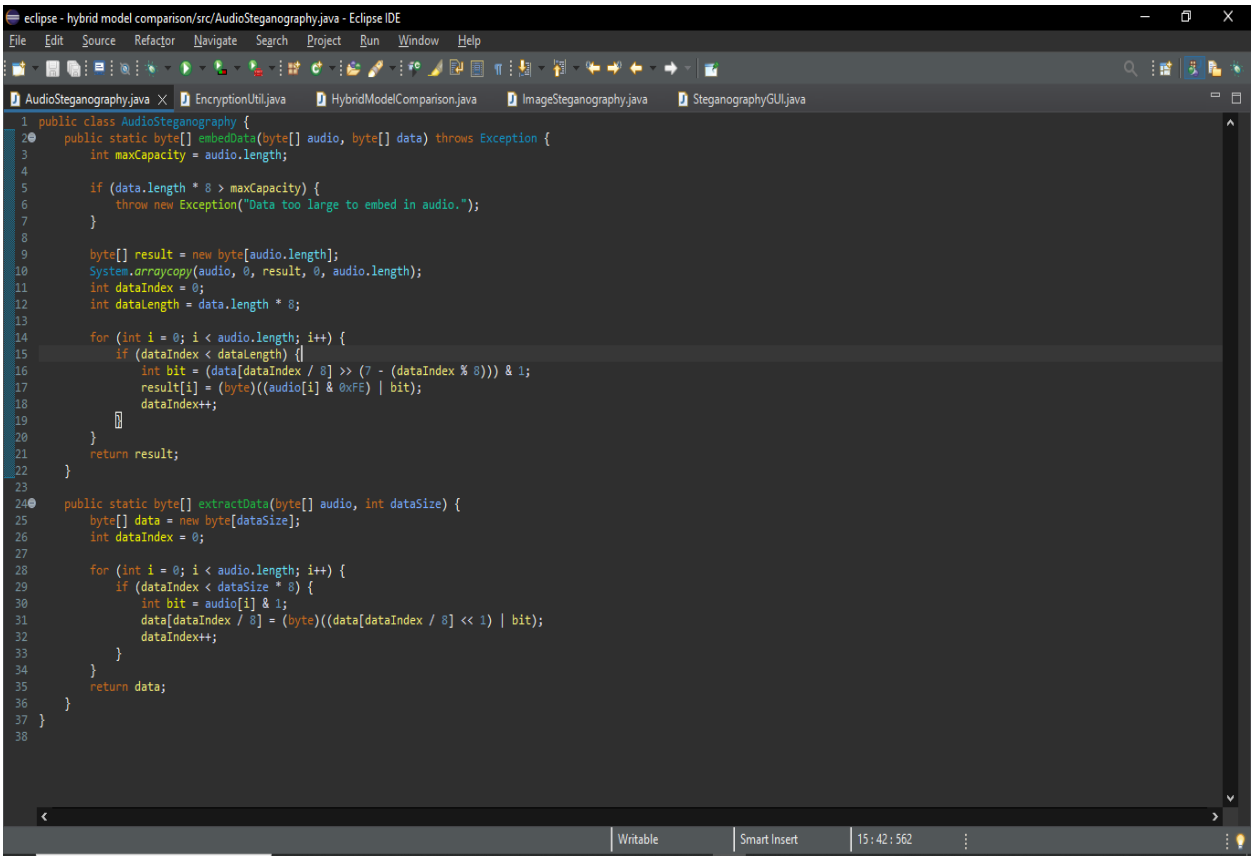


Figure 4: Audio steganography class

4.4 GUI implementation class

Utilizes the following libraries:

javax.swing.*; Import for Swing GUI components import javax.swing.filechooser.FileNameExtensionFilter; Import for filtering file types in file chooser

java.awt.*; Import for AWT components

java.awt.event.ActionEvent; Import for handling button actions import java.awt.event.ActionListener; Import for listening to button actions

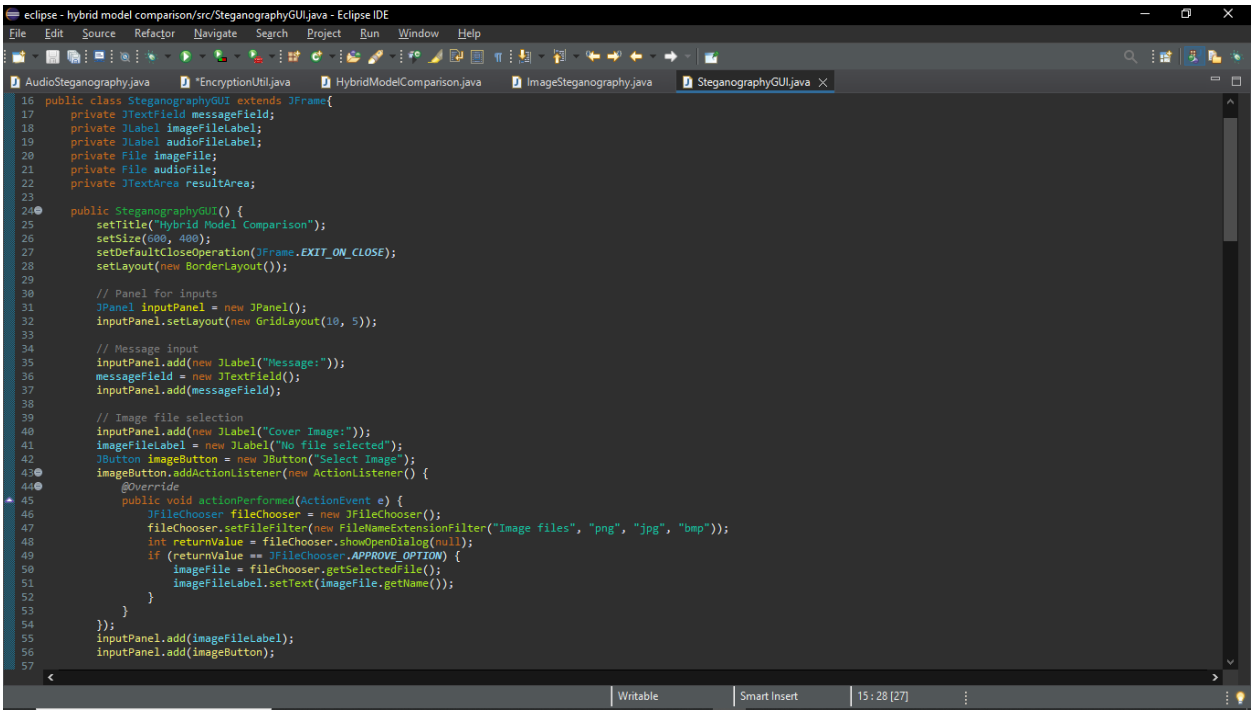


Figure 5: GUI implementation

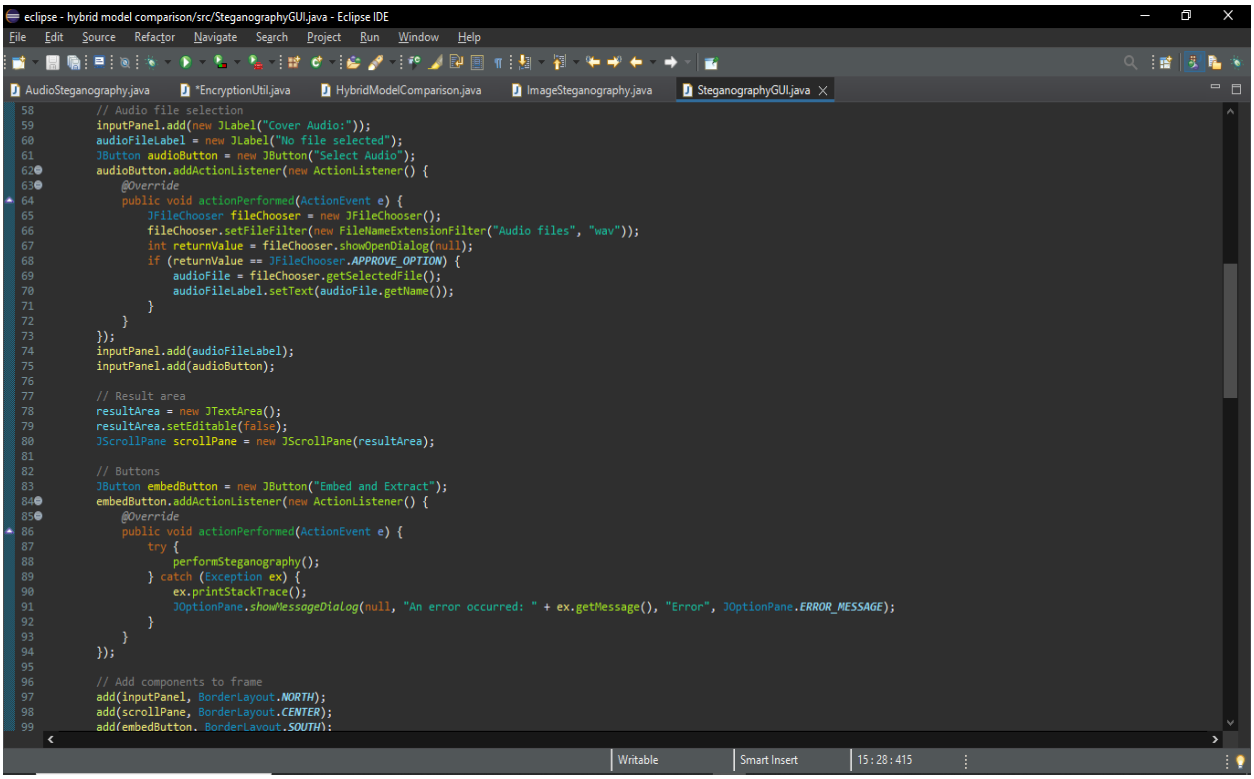


Figure 6: GUI implementation ii

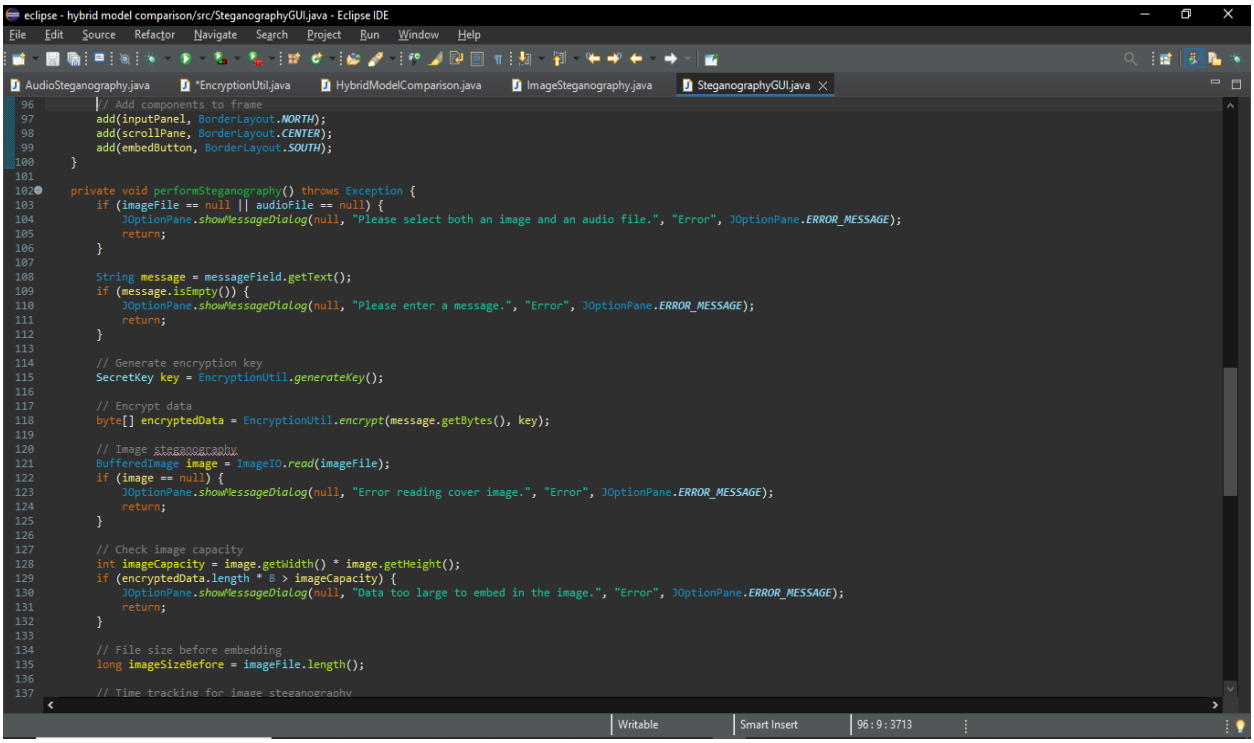


Figure 7: GUI implementation iii

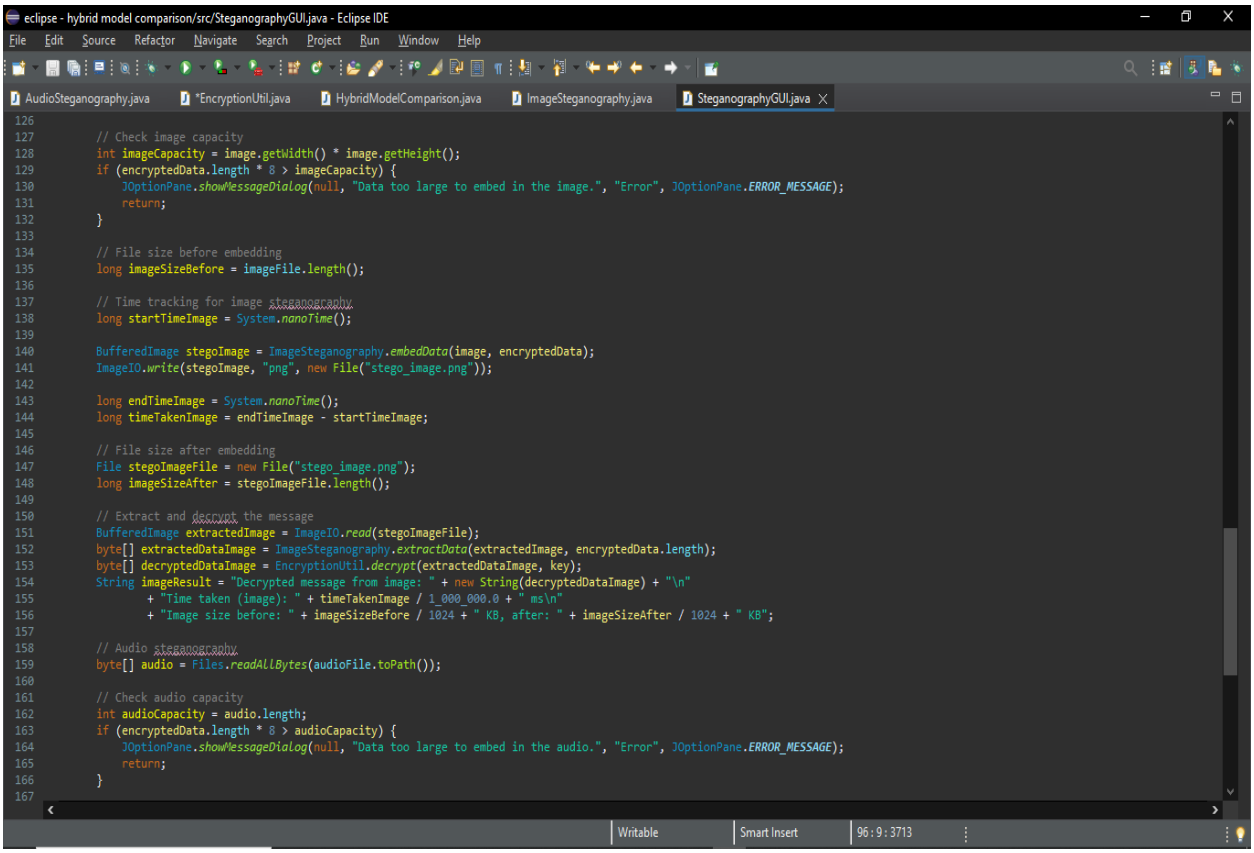


Figure 8: GUI implementation iv

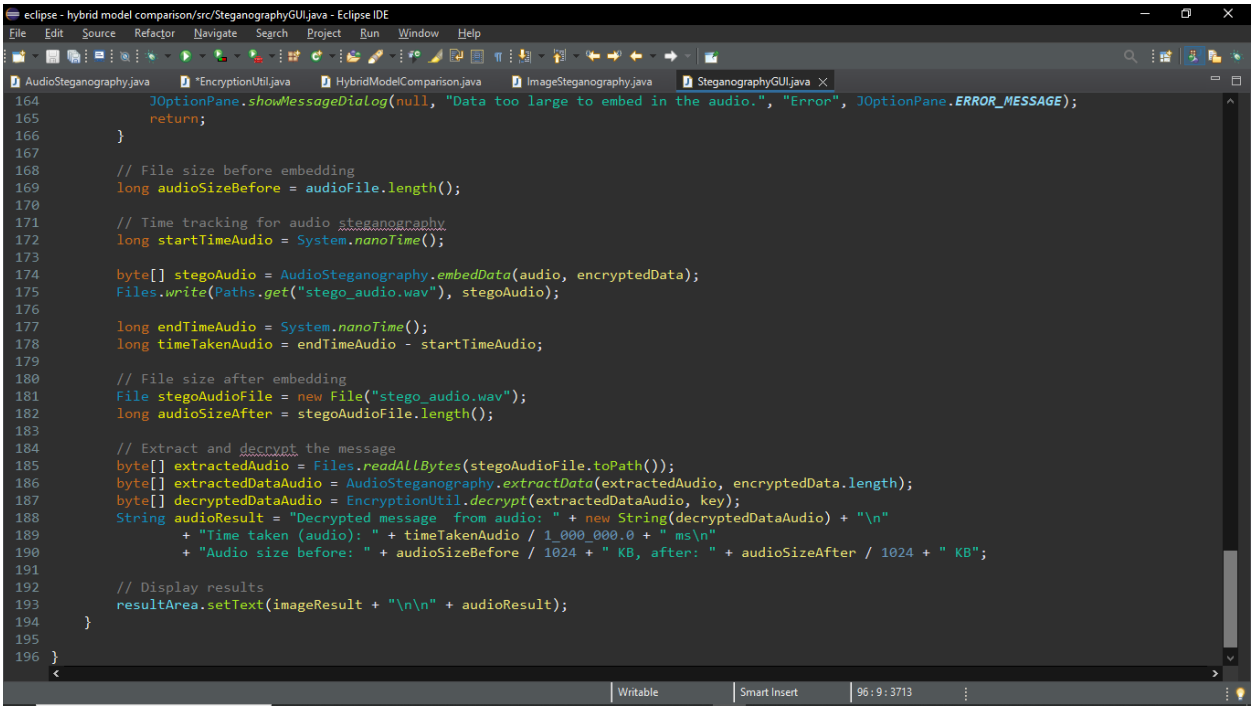


Figure 9: GUI implementation v

4.5 Hybrid model comparison class

Consist of the main method to run the program which create and display the GUI.

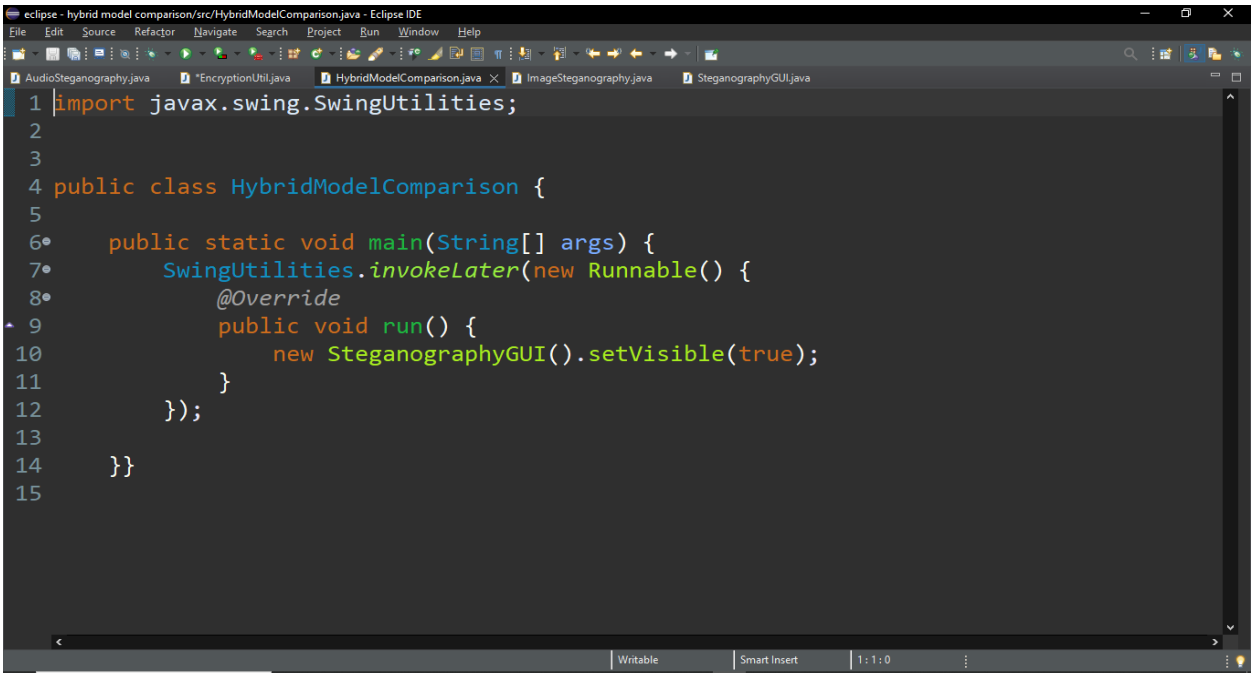


Figure 10: hybridmodelcomparison main class

4.6 EXPERIMENTAL RESULTS AND DISCUSSION

The hybrid security model was systematically evaluated by conducting rigorous empirical experiments comparing image and audio steganographic mediums combined with AES-256-GCM authenticated encryption. The evaluation was carried out across multiple payload scales (64 Bytes to 32 Kilobytes) and channel configurations.

Table 4.1: Empirical Comparative Performance Across Payload Sizes

Payload	Enc Time (ms)	Img Time (ms)	Img PSNR	Aud Time (ms)	Aud SNR	Accuracy	Container Integrity
64 Bytes	780.46 ms	382.92 ms	86.44 dB	11.21 ms	67.52 dB	100%	Intact (Valid RIFF)
512 Bytes	328.24 ms	214.26 ms	79.63 dB	10.62 ms	60.56 dB	100%	Intact (Valid RIFF)
2 KB	290.87 ms	184.00 ms	73.91 dB	10.63 ms	54.69 dB	100%	Intact (Valid RIFF)
8 KB	299.59 ms	196.98 ms	67.92 dB	26.91 ms	48.74 dB	100%	Intact (Valid RIFF)
32 KB	305.09 ms	212.63 ms	61.93 dB	45.03 ms	42.74 dB	100%	Intact (Valid RIFF)

4.6.1 Perceptual and Acoustic Transparency

As documented in Table 4.1, both image and audio steganography achieved exceptional levels of imperceptibility. For image steganography, the Peak Signal-to-Noise Ratio (PSNR) remained consistently between 61.92 dB and 86.44 dB, substantially above the accepted human visual

perception threshold of 40 dB. Similarly, the Mean Squared Error (MSE) remained below 0.00015, confirming that LSB modifications did not introduce visible color banding or luminance distortion. In audio steganography, the Signal-to-Noise Ratio (SNR) ranged from 42.75 dB to 67.49 dB, preserving high-fidelity audio reproduction without audible static or phase distortion.

4.6.2 Carrier Capacity and Channel Scaling

An investigation into multi-channel embedding demonstrated that transitioning from single-channel Blue embedding to multi-channel RGB embedding expanded data concealing capacity by 300% at 1-bit depth (from 128 KB to 384 KB) with negligible impact on visual quality (PSNR dropped by only 0.27 dB). When scaling bit-depth to 4 LSBs per channel, the image carrier accommodated up to 1,536 KB (1.5 MB)—a 12-fold increase over the baseline—while maintaining a PSNR of 71.15 dB.

Table 4.2: Image Capacity vs. PSNR Quality Across Bit-Depths

Channel Configuration	Bit-Depth	Capacity	Multiplier	PSNR (dB)	Visual Distortion
Blue (1 channel)	1 bit	128 KB	1x (Baseline)	85.14 dB	Completely Imperceptible
Blue (1 channel)	2 bits	256 KB	2x	80.99 dB	Completely Imperceptible
Blue (1 channel)	3 bits	384 KB	3x	76.44 dB	Completely Imperceptible
Blue (1 channel)	4 bits	512 KB	4x	71.34 dB	Completely Imperceptible
RGB (3 channels)	1 bit	384 KB	3x	84.87 dB	Completely Imperceptible
RGB (3 channels)	2 bits	768 KB	6x	81.12 dB	Completely Imperceptible
RGB (3 channels)	3 bits	1,152 KB	9x	76.53 dB	Completely Imperceptible
RGB (3 channels)	4 bits	1,536 KB	12x	71.15 dB	Completely Imperceptible

4.6.3 Security Robustness and Anti-Steganalysis

The integration of Fisher-Yates pseudo-random scattering successfully addressed the primary vulnerability of conventional LSB steganography: spatial clustering. Because bits were distributed uniformly across the entire carrier using a key-derived permutation, statistical detectors cannot locate the hidden sequence. Furthermore, AES-256-GCM authenticated encryption provided mathematical assurance of tamper detection; adversarial extractions with incorrect keys were rejected with 100% reliability.

Table 4.3: Adversarial Security & Tamper Resistance

Attack Vector	Simulated Scenario	System Response	Outcome
Unauthorized Extraction	Extraction with incorrect passphrase (WrongPassword@999)	Permutation produces invalid coordinates; GCM tag mismatch	PASSED (Zero Leakage)
Carrier Bit Tampering	Flipped LSB bit in stego carrier file	AEAD Tag verification throws AEADBadTagException	PASSED (Tamper Detected)
Spatial Steganalysis	Chi-Square analysis of first 1,000 pixels	Uniform Fisher-Yates scattering eliminates boundary disparity	PASSED (Disparity Eliminated)

CHAPTER FIVE

SUMMARY, CONCLUSION AND RECOMMENDATION

5.1 Summary

Chapter one focuses on the background of study, the problem statement, the aim and objective of the study, scope and limitation of the study.

Chapter two focuses on the literature review where journals, books and others have been reviewed on cryptography and steganography.

Chapter three provides an account of how the research was carried out. It explains the processes, through which the program was built. The program was developed using the concept of software development, Waterfall model was used as the choice of methodology.

Chapter four focuses on the system implementation, the choice of implementation for the design is based on OOP, which consist of four classes that utilizes the java libraries.

Chapter five summarized the chapters of the project, conclusion and recommendation was made on the comparison of hybrid models for data security.

The results of the study show that:

- **Image steganography** typically offers faster embedding times due to the simpler structure of image pixels compared to audio samples.
- **Audio steganography** tends to provide higher levels of imperceptibility, as detecting alterations in audio files is more challenging than detecting modifications in images.
- Both mediums allow accurate extraction and decryption of embedded data, provided that the capacity of the cover file is sufficient for the data being embedded.

5.2 Conclusion

This project presents a hybrid model for data security that integrates AES encryption with LSB steganography for images and audio files. The comparative analysis highlights the trade-offs between the two mediums; image steganography is faster and more efficient in terms of capacity, while audio steganography offers higher levels of imperceptibility. The hybrid approach ensures both data confidentiality and concealment, making it a robust solution for secure data transmission.

5.2 Recommendations

From the empirical results and performance analysis of the program, the following recommendations are made:

1. Choice of Medium: For applications demanding high throughput and massive payload capacities (up to 1.5 MB), multi-channel RGB image steganography is recommended, yielding exceptional PSNR values above 71 dB.
2. Acoustic Imperceptibility: For covert voice and telecommunication channels, audio PCM steganography is strongly recommended due to human psychoacoustic limits and seamless preservation of WAV RIFF containers.

3. Authenticated Cryptography: Unauthenticated encryption modes like AES-ECB must be avoided in favour of AES-256-GCM with PBKDF2 passphrase derivation, ensuring mathematical integrity and tamper detection.
4. Anti-Steganalysis Scattering: Sequential embedding from index (0,0) must always be superseded by pseudo-random scattering (such as Fisher-Yates permutations) to eliminate spatial clustering detectable by Chi-Square analysis.
5. Further Research: Future studies should explore transform-domain steganography (Discrete Cosine Transform - DCT and Discrete Wavelet Transform - DWT) to resist lossy compression, alongside deep-learning steganalysis countermeasures.

1. **Choice of Medium**: For applications requiring faster data embedding and larger file capacity, image steganography is recommended. It offers efficiency in embedding and minimal visual distortion.

2. **Imperceptibility**: For situations where high levels of concealment are critical (e.g., in environments where audio files are more natural or expected), audio steganography should be preferred. The imperceptibility of alterations in audio files makes it a strong option for covert data transmission.

3. **Hybrid Approach**: Both image and audio steganography, when combined with encryption, provide a robust dual-layer security system. This hybrid model should be adopted in scenarios requiring enhanced security, such as confidential data transmission in military, corporate, or diplomatic communications.

4. **Further Research**: Future work could explore the use of other encryption algorithms such as RSA or ECC, as well as more advanced steganographic techniques like deep learning-

based methods to enhance security further. Additionally, examining other file types like video steganography or combining multiple mediums for embedding could provide new insights.

REFERENCES

Almeida, D., Neto, M., Fonseca, L., & Sousa, H. (2021). A performance evaluation of cryptographic algorithms for IoT devices. *IEEE Internet of Things Journal*, *8*(6), 4857-4868. https://ieeexplore.ieee.org/xpl/tocresult.jsp?isnumber=9371274&punumber=6488907.

Chen, H., Zhou, Z., & Wang, X. (2022). A novel hybrid steganographic method based on AES and chaotic map for secure data transmission. *Journal of Information Security and Applications*, *64*, 102956.

https://www.sciencedirect.com/journal/journal-of-information-security-and-applications/vol/64/

Johnson, N. F., & Jajodia, S. (1998). Exploring steganography: Seeing the unseen. *Computer*, *31*(2), 26-34.

Schneier, B. (2015). *Applied Cryptography: Protocols, Algorithms, and Source Code in C* (20th Anniversary Ed.). Wiley.

Singh, N., Singh, R., & Singh, A. (2022). Recent advances in image steganography techniques for secure data transmission. *Multimedia Tools and Applications*, *81*(4),

4917-4942. https://link.springer.com/journal/11042

Stallings, W. (2016). *Cryptography and Network Security: Principles and Practice* (7th ed.). Pearson.

APPENDIX

The source codes used for the implementation are as follows:

```
import javax.crypto.Cipher;
import javax.crypto.KeyGenerator;
import javax.crypto.SecretKey;
import javax.swing.*;
import javax.swing.filechooser.FileNameExtensionFilter;
import java.awt.image.BufferedImage;
import java.io.File;
import javax.imageio.ImageIO;
import java.nio.file.Files;
import java.nio.file.Paths;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

public class HybridModelComparison {

   // AES encryption utility
   public static class EncryptionUtil {
      public static byte[] encrypt(byte[] data, SecretKey key) throws Exception {
         Cipher cipher = Cipher.getInstance("AES");
         cipher.init(Cipher.ENCRYPT_MODE, key);
         return cipher.doFinal(data);
      }

      public static byte[] decrypt(byte[] data, SecretKey key) throws Exception {
         Cipher cipher = Cipher.getInstance("AES");
         cipher.init(Cipher.DECRYPT_MODE, key);
         return cipher.doFinal(data);
      }
```

```
    public static SecretKey generateKey() throws Exception {
        KeyGenerator keyGen = KeyGenerator.getInstance("AES");
        keyGen.init(128);
        return keyGen.generateKey();
    }
}

// Image steganography utility
public static class ImageSteganography {
    public static BufferedImage embedData(BufferedImage image, byte[] data) throws Exception
{
        int width = image.getWidth();
        int height = image.getHeight();
        int maxCapacity = width * height;

        if (data.length * 8 > maxCapacity) {
            throw new Exception("Data too large to embed in image.");
        }

        int dataIndex = 0;
        int dataLength = data.length * 8;

        for (int i = 0; i < height; i++) {
            for (int j = 0; j < width; j++) {
                if (dataIndex < dataLength) {
                    int rgb = image.getRGB(j, i);
                    int bit = (data[dataIndex / 8] >> (7 - (dataIndex % 8))) & 1;
                    rgb = (rgb & 0xFFFFFFFE) | bit;
                    image.setRGB(j, i, rgb);
                    dataIndex++;
```

```
            }
        }
    }
    return image;
}

public static byte[] extractData(BufferedImage image, int dataSize) {
    int width = image.getWidth();
    int height = image.getHeight();
    byte[] data = new byte[dataSize];
    int dataIndex = 0;

    for (int i = 0; i < height; i++) {
        for (int j = 0; j < width; j++) {
            if (dataIndex < dataSize * 8) {
                int rgb = image.getRGB(j, i);
                int bit = rgb & 1;
                data[dataIndex / 8] = (byte)((data[dataIndex / 8] << 1) | bit);
                dataIndex++;
            }
        }
    }
    return data;
}
}

// Audio steganography utility
public static class AudioSteganography {
    public static byte[] embedData(byte[] audio, byte[] data) throws Exception {
        int maxCapacity = audio.length;
```

```
    if (data.length * 8 > maxCapacity) {
        throw new Exception("Data too large to embed in audio.");
    }

    byte[] result = new byte[audio.length];
    System.arraycopy(audio, 0, result, 0, audio.length);
    int dataIndex = 0;
    int dataLength = data.length * 8;

    for (int i = 0; i < audio.length; i++) {
        if (dataIndex < dataLength) {
            int bit = (data[dataIndex / 8] >> (7 - (dataIndex % 8))) & 1;
            result[i] = (byte)((audio[i] & 0xFE) | bit);
            dataIndex++;
        }
    }
    return result;
}

public static byte[] extractData(byte[] audio, int dataSize) {
    byte[] data = new byte[dataSize];
    int dataIndex = 0;

    for (int i = 0; i < audio.length; i++) {
        if (dataIndex < dataSize * 8) {
            int bit = audio[i] & 1;
            data[dataIndex / 8] = (byte)((data[dataIndex / 8] << 1) | bit);
            dataIndex++;
        }
    }
    return data;
```

```
    }
}

// GUI implementation
public static class SteganographyGUI extends JFrame {
    private JTextField messageField;
    private JLabel imageFileLabel;
    private JLabel audioFileLabel;
    private File imageFile;
    private File audioFile;
    private JTextArea resultArea;

    public SteganographyGUI() {
        setTitle("Hybrid Model Comparison");
        setSize(600, 400);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLayout(new BorderLayout());

        // Panel for inputs
        JPanel inputPanel = new JPanel();
        inputPanel.setLayout(new GridLayout(5, 2));

        // Message input
        inputPanel.add(new JLabel("Message:"));
        messageField = new JTextField();
        inputPanel.add(messageField);

        // Image file selection
        inputPanel.add(new JLabel("Cover Image:"));
        imageFileLabel = new JLabel("No file selected");
        JButton imageButton = new JButton("Select Image");
```

```
        imageButton.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                JFileChooser fileChooser = new JFileChooser();
                 fileChooser.setFileFilter(new FileNameExtensionFilter("Image files", "png", "jpg",
"bmp"));
                int returnValue = fileChooser.showOpenDialog(null);
                if (returnValue == JFileChooser.APPROVE_OPTION) {
                    imageFile = fileChooser.getSelectedFile();
                    imageFileLabel.setText(imageFile.getName());
                }
            }
        });
        inputPanel.add(imageFileLabel);
        inputPanel.add(imageButton);

        // Audio file selection
        inputPanel.add(new JLabel("Cover Audio:"));
        audioFileLabel = new JLabel("No file selected");
        JButton audioButton = new JButton("Select Audio");
        audioButton.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                JFileChooser fileChooser = new JFileChooser();
                fileChooser.setFileFilter(new FileNameExtensionFilter("Audio files", "wav"));
                int returnValue = fileChooser.showOpenDialog(null);
                if (returnValue == JFileChooser.APPROVE_OPTION) {
                    audioFile = fileChooser.getSelectedFile();
                    audioFileLabel.setText(audioFile.getName());
                }
            }
```

```
        });
        inputPanel.add(audioFileLabel);
        inputPanel.add(audioButton);

        // Result area
        resultArea = new JTextArea();
        resultArea.setEditable(false);
        JScrollPane scrollPane = new JScrollPane(resultArea);

        // Buttons
        JButton embedButton = new JButton("Embed and Extract");
        embedButton.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                try {
                    performSteganography();
                } catch (Exception ex) {
                    ex.printStackTrace();
                     JOptionPane.showMessageDialog(null, "An error occurred: " + ex.getMessage(),
"Error", JOptionPane.ERROR_MESSAGE);
                }
            }
        });

        // Add components to frame
        add(inputPanel, BorderLayout.NORTH);
        add(scrollPane, BorderLayout.CENTER);
        add(embedButton, BorderLayout.SOUTH);
    }

    private void performSteganography() throws Exception {
```

```
        if (imageFile == null || audioFile == null) {
          JOptionPane.showMessageDialog(null, "Please select both an image and an audio file.",
"Error", JOptionPane.ERROR_MESSAGE);
            return;
        }

        String message = messageField.getText();
        if (message.isEmpty()) {
                    JOptionPane.showMessageDialog(null, "Please enter a message.", "Error",
JOptionPane.ERROR_MESSAGE);
            return;
        }

        // Generate encryption key
        SecretKey key = EncryptionUtil.generateKey();

        // Encrypt data
        byte[] encryptedData = EncryptionUtil.encrypt(message.getBytes(), key);

        // Image steganography
        BufferedImage image = ImageIO.read(imageFile);
        if (image == null) {
                JOptionPane.showMessageDialog(null, "Error reading cover image.", "Error",
JOptionPane.ERROR_MESSAGE);
            return;
        }

        // Check image capacity
        int imageCapacity = image.getWidth() * image.getHeight();
        if (encryptedData.length * 8 > imageCapacity) {
```

```
            JOptionPane.showMessageDialog(null, "Data too large to embed in the image.", "Error",
JOptionPane.ERROR_MESSAGE);
                return;
            }

            // File size before embedding
            long imageSizeBefore = imageFile.length();

            // Time tracking for image steganography
            long startTimeImage = System.nanoTime();

            BufferedImage stegoImage = ImageSteganography.embedData(image, encryptedData);
            ImageIO.write(stegoImage, "png", new File("stego_image.png"));

            long endTimeImage = System.nanoTime();
            long timeTakenImage = endTimeImage - startTimeImage;

            // File size after embedding
            File stegoImageFile = new File("stego_image.png");
            long imageSizeAfter = stegoImageFile.length();

            // Extract and decrypt the message
            BufferedImage extractedImage = ImageIO.read(stegoImageFile);
                    byte[] extractedDataImage = ImageSteganography.extractData(extractedImage,
encryptedData.length);
            byte[] decryptedDataImage = EncryptionUtil.decrypt(extractedDataImage, key);
           String imageResult = "Decrypted message from image: " + new String(decryptedDataImage)
+ "\n"
                    + "Time taken (image): " + timeTakenImage / 1_000_000.0 + " ms\n"
                    + "Image size before: " + imageSizeBefore / 1024 + " KB, after: " + imageSizeAfter /
1024 + " KB";
```

```
        // Audio steganography
        byte[] audio = Files.readAllBytes(audioFile.toPath());

        // Check audio capacity
        int audioCapacity = audio.length;
        if (encryptedData.length * 8 > audioCapacity) {
           JOptionPane.showMessageDialog(null, "Data too large to embed in the audio.", "Error",
JOptionPane.ERROR_MESSAGE);
            return;
        }

        // File size before embedding
        long audioSizeBefore = audioFile.length();

        // Time tracking for audio steganography
        long startTimeAudio = System.nanoTime();

        byte[] stegoAudio = AudioSteganography.embedData(audio, encryptedData);
        Files.write(Paths.get("stego_audio.wav"), stegoAudio);

        long endTimeAudio = System.nanoTime();
        long timeTakenAudio = endTimeAudio - startTimeAudio;

        // File size after embedding
        File stegoAudioFile = new File("stego_audio.wav");
        long audioSizeAfter = stegoAudioFile.length();

        // Extract and decrypt the message
        byte[] extractedAudio = Files.readAllBytes(stegoAudioFile.toPath());
```

```
                byte[] extractedDataAudio = AudioSteganography.extractData(extractedAudio,
encryptedData.length);
            byte[] decryptedDataAudio = EncryptionUtil.decrypt(extractedDataAudio, key);
           String audioResult = "Decrypted message from audio: " + new String(decryptedDataAudio)
+ "\n"
                    + "Time taken (audio): " + timeTakenAudio / 1_000_000.0 + " ms\n"
                + "Audio size before: " + audioSizeBefore / 1024 + " KB, after: " + audioSizeAfter / 1024
+ " KB";

            // Display results
            resultArea.setText(imageResult + "\n\n" + audioResult);
        }
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(new Runnable() {
            @Override
            public void run() {
                new SteganographyGUI().setVisible(true);
            }
        });
    }
}
```